

Time as Intervals, Not Instants: A Practical Implementation of Allen’s Interval Algebra Over ISO 8601

Kip Cole ✉

Independent

Abstract

Mainstream programming languages model temporal values as scalar instants: a `Date` is a point, a `DateTime` is a point, and a partial value such as “June 2026” has no canonical representation at all. Yet the temporal values that humans, calendars, and business logic actually manipulate are spans on the timeline — the year 2026, the afternoon of the 15th, an archaeological period, a meeting. `Tempo` is an Elixir library that reverses the polarity: every temporal value is a bounded half-open interval at a declared resolution, set-theoretic combination is closed under that type, and comparison is defined by Allen’s thirteen relations rather than by scalar ordering. The result eliminates an entire class of off-by-one and “end-of-day” bugs by construction, gives partial ISO 8601 values (2026, 2026-06, 2026-06-XX) an unambiguous denotational semantics, and lifts Allen’s algebra from a textbook abstraction to an everyday predicate vocabulary (`overlaps?`, `within?`, `adjacent?`). This extended abstract describes the conceptual model, the unification of partial dates with explicit intervals, and the design of set operations over intervals and interval sets that is the library’s current focus.

2012 ACM Subject Classification Information systems → Temporal data; Software and its engineering → Domain specific languages; Theory of computation → Modal and temporal logics

Keywords and phrases Temporal intervals, Allen’s interval algebra, ISO 8601, calendrical systems, partial dates, set operations on time, domain-specific languages

Digital Object Identifier 10.4230/OASICS...

Related Version Source repository: <https://github.com/kipcole9/tempo>

1 Motivation: the instant–span impedance mismatch

Every mainstream temporal library — Python’s `datetime`, Java’s `java.time`, JavaScript’s `Temporal` proposal, Elixir’s own `Date/Time/DateTime` — shares a design choice that goes largely unexamined: a temporal value is a *scalar instant* on the timeline. A day is the midnight at its start, a month is the midnight on its first, a year is the midnight on January 1. The span that humans actually mean when they say “June 2026” is materialised, if at all, by ad-hoc helpers (`start_of_month`, `end_of_day`) that each project re-implements with its own conventions about inclusivity, time zones, and DST.

This instant-first model has three persistent consequences:

Ambiguity at the type level.

The expression `Date.new(2026, 6, 15)` denotes a 24-hour span in business prose and an instant in the type system. The disagreement is resolved informally, per call site, and is a recurring source of off-by-one bugs at month, year, and DST boundaries. Tiwari et al.’s recent empirical study of 151 date/time bugs in open-source Python projects identifies time-zone-related mistakes as the largest single category, with most bugs traceable to misconceptions about library API conventions and edge-case behaviour [7]; Liva’s earlier work on Java programs reaches a structurally similar conclusion [8].



© Kip Cole;

licensed under Creative Commons License CC-BY 4.0

OpenAccess Series in Informatics

OASICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

42 **No representation for partial dates.**

43 A user who writes “2022” or “the 1560s” or “approximately June 2026” has produced a
 44 perfectly meaningful temporal value, but the type system has no home for it. Solutions
 45 range from sentinel values (`Date.new(2022, 1, 1)` stands in for “the year”) to parallel type
 46 hierarchies (EDTF [5]) that exist outside the standard library, or to dedicated calendric type
 47 systems such as Bry and Spranger’s CaTTS for Semantic Web query languages [9]. The
 48 fragmentation is structural, not incidental.

49 **Comparison is reduced to scalar ordering.**

50 Two dates are either $<$, $=$, or $>$. The richer relational structure that Allen identified in 1983 [1]
 51 — *precedes*, *meets*, *overlaps*, *starts*, *during*, *finishes*, *equals*, and their inverses — is invisible at
 52 the type level, and reimplemented case-by-case in business logic that interrogates whether a
 53 meeting “overlaps” the workday or a contract “contains” a billing period.

54 Tempo’s design is the observation that all three problems dissolve simultaneously if
 55 intervals are made the primitive type. There are no points: an apparent “instant” is the
 56 interval of one unit at the finest declared resolution, in the same spirit as the time ontology
 57 $T_{\text{bounded_meeting}}$ of Grüninger and Li [10], in which intervals are the only entities in the
 58 domain and Allen’s relations are derivable from a single primitive (*meets*).

59 **Empirical coverage.**

60 Against the 151 Python bugs catalogued by Tiwari et al., the design choices described
 61 in the remainder of this paper address every in-scope bug class: 43 (28%) are *avoided by*
 62 *construction* (the naïve / aware datetime split has no Tempo analogue), 56 (37%) are *directly*
 63 *addressed* by explicit machinery (zone propagation through set operations, DST gap rejection
 64 at parse time and IXDTF-offset fold disambiguation per RFC 9557 §4.5, calendar-aware
 65 arithmetic, half-open materialisation, microsecond-precision durations), 28 (19%) are partial,
 66 and 24 (16%) are out of scope (Python deprecations and third-party-library logic such as
 67 holiday tables). *Zero* bugs are classified as architectural gaps. The per-bug classification is
 68 provided as supplementary material at the repository.

69 **2 Intervals as the primitive**

70 Every Tempo value is a bounded half-open interval $[first, last)$ at a declared resolution. Inter-
 71 nally an interval is a pair of endpoints, which is the standard finite model of $T_{\text{bounded_meeting}}$
 72 characterised by Grüninger and Li [10] (their Theorem 15): vertices of the directed graph
 73 $\mathfrak{M}^{\text{bounded_meeting}}$ correspond to intervals, edges to the *meets* relation, and structures up to
 74 isomorphism faithfully capture the ontology’s models.

75 A note on terminology before proceeding: *resolution* as used throughout this paper is
 76 a Tempo-side property of an input value (2026 carries year-resolution, 2026-06-15 carries
 77 day-resolution) used for partial-date materialisation, default iteration, and display. It is
 78 *not* the granularity of Bettini, Wang and Jajodia [6] (a lattice of partitions of the time line
 79 that participates in Allen comparisons), and it does *not* appear in $T_{\text{bounded_meeting}}$, where
 80 intervals are individuated solely by their position in the *meets*-structure. Once an interval
 81 is materialised to its $[first, last)$ form, resolution plays no further role in Allen comparisons
 82 or in set-theoretic operations. The word is also distinct from *precision* and *accuracy* in
 83 the measurement-science sense: Tempo makes no claim about the precision of a value’s
 84 construction, only about the extent it denotes.

A second Tempo-side distinction is between *anchored* and *non-anchored* values. An anchored value carries a year component (e.g. 2026-06-15 or 2026); materialising it yields an interval that is an *individual* in $T_{\text{bounded_meeting}}$ in the standard model-theoretic sense — a vertex of $\mathfrak{M}^{\text{bounded_meeting}}$ (Theorem 15) with a definite position in the *meets*-structure. A non-anchored value (a time-of-day template such as T10:30) is not an individual: it is an open expression with the date components as free variables, denoting the family of individuals that result when those variables are bound. Tempo’s `anchor/2` and `align/3 :bound` option supply that binding; the ontological move is *instantiation*, and the result is the intervalised individual the ontology talks about. The two operations Tempo exposes for working with non-anchored values are *projection* (resolving to a specific universal-timeline extent given an anchor) and *selection* (filtering a set of candidate anchors). Neither has a counterpart inside $T_{\text{bounded_meeting}}$ itself, which is silent on schemas; they sit, like resolution, in the engineering layer above.

The library distinguishes two forms of interval that interconvert losslessly.

Implicit spans.

A single ISO 8601 datetime *is* an interval whose span is determined by the next-higher resolution that is not stated:

2026	≡	[2026-01-01, 2027-01-01)
2026-01	≡	[2026-01-01, 2026-02-01)
2026-01-15	≡	[2026-01-15, 2026-01-16)
2026-01-15T10	≡	[2026-01-15T10:00, 2026-01-15T11:00)

The partial-date problem disappears: 2026 is not an under-specified instant, it is a definite interval of one year (the distinction is semantic, not a claim about measurement precision or accuracy). The “last day of the month” problem disappears: the upper bound of 2026-01 is 2026-02-01, computed from the calendar arithmetic of the target calendar (Gregorian, Hebrew, Islamic Civil, ...), and is correct by construction in leap years and across month-length variation.

Explicit spans.

A pair of datetimes written with a range operator — 2026-01-01..2026-02-01 or, in IXDTF-extended form [4], 2026-01-01/2026-02-01 — carries its own boundaries. Its iteration unit is the resolution of its boundaries, not of a partial denotation.

Half-open by design.

The half-open $[first, last)$ convention is not notational shorthand; it is enforced at every level of the API. Adjacent intervals concatenate cleanly: $[a, b) \cup [b, c) = [a, c)$ with neither overlap nor gap. Iteration yields a sequence whose union is exactly the parent interval. Conversion between implicit and explicit forms is a structural identity, not an arithmetic approximation. Library code that treats the upper bound as inclusive is, by the design contract, a defect. Alternative timestamp representations have been explored at TIME — notably Dyreson and Ahsan’s log-segmented timestamps [11, 12], which decompose a period into power-of-two segments so that standard B^+ -tree indexes can support sequenced and nonsequenced queries on an unmodified DBMS. The two designs occupy different points in the same space: log-segmented timestamps optimise query evaluation in a relational engine; Tempo’s half-open intervals optimise expressiveness and uniformity at the application layer.

125 3 Allen’s algebra as the predicate vocabulary

126 Tempo exposes Allen’s thirteen relations [1] directly as the result of its core comparison
 127 primitive. The first-order ontology underlying the algebra is characterised by Grüninger and
 128 Li [10], who identify the time ontology logically synonymous with Allen’s interval algebra
 129 and prove a representation theorem for its models; Tempo is, in effect, an ergonomic software
 130 realisation of that ontology over the calendar-aware datetime values defined by ISO 8601.
 131 The thirteen relations are returned directly from the relation primitive:

```
132 Tempo.relation(a, b) ::
133     :precedes | :meets | :overlaps | :finished_by |
134     :contains | :starts | :equals |
135     :started_by | :during | :finishes | :overlapped_by |
136     :met_by | :preceded_by
```

137 This is rarely the right surface for application code, however — business questions tend
 138 to name unions of Allen relations. “Does the meeting fall within the workday?” is the set
 139 $\{equals, starts, during, finishes\}$; “does this period touch that one?” is the complement of
 140 $\{precedes, preceded_by\}$. Hand-rolled inline matching on relation lists is verbose and obscures
 141 the intent, so the library promotes the common unions to first-class predicates — `within?/2`,
 142 `overlaps?/2`, `adjacent?/2`, `contains?/2` — each defined as a specific subset of Allen’s
 143 relations and each readable aloud as the English question it answers.

144 The predicate vocabulary is not decorative. It is a design discipline: when an example
 145 in the cookbook reaches for a relation list or a manual endpoint comparison, that is taken
 146 as evidence that the abstraction is missing, and the predicate is added before the example
 147 is written. The same discipline applies to durations (`at_least?`, `shorter_than?`) and to
 148 interval-set queries (`any_overlaps?`, `disjoint?`). The library’s public surface is, deliberately,
 149 an English-prose description of Allen’s algebra over ISO 8601 values.

150 4 Set operations on intervals and interval sets

151 The current implementation focus is closing the library under set-theoretic operations. The
 152 intuition is straightforward — any two intervals can be combined into a (possibly disjoint,
 153 possibly zero-member) set of intervals — but the engineering requires careful handling of the
 154 implicit/explicit-span distinction, of cross-calendar operands, and of cross-zone arithmetic. A
 155 single connected, non-empty result is returned as an `Interval`; every other shape — disjoint,
 156 or zero-member — is returned as an `IntervalSet`. An interval is never returned with zero
 157 extent, in line with $T_{bounded_meeting}$ ’s exclusion of degenerate intervals from the domain.

158 Pairwise operations.

159 `union/2`, `intersection/2`, and `difference/2` on a pair of operands a, b return either an
 160 interval or an `IntervalSet`, depending on whether the result is connected. Operands may
 161 be supplied in either implicit or explicit form; the library normalises both to the explicit
 162 half-open representation, performs the geometric operation, and re-presents the result in the
 163 most compact form (a single interval when possible).

164 n -ary operations and canonicalisation.

165 Extending the pairwise operations to lists of intervals raises the question of canonical form.
 166 Tempo defines the canonical form of an `IntervalSet` as the sorted, coalesced sequence in

167 which no two intervals overlap or meet. Coalescing is the closure of the union operation under
 168 iteration: $\bigcup_i [a_i, b_i]$ expressed as a minimal disjoint sequence. This is the form returned by
 169 every set-producing operation in the public API. Coalescing also realises the Sum Axiom of
 170 $T_{\text{bounded_meeting}}$ (axiom 15 in [10]): three chain-meeting intervals $\text{meets}(i, j) \wedge \text{meets}(j, k) \wedge$
 171 $\text{meets}(k, m)$ are summed into the single interval that runs from i 's start to m 's end.

172 Cross-calendar and cross-zone semantics.

173 Because each operand carries its own calendar and zone, set operations are defined on the
 174 underlying UTC reference frame — each endpoint is projected to gregorian seconds (the
 175 standard Erlang representation) so that endpoints from different calendars or zones can be
 176 ordered in a single linear sequence. The word “instant” is avoided here because Grüninger
 177 and Li's ontology excludes points from its domain (intervals are the only entities); naming
 178 the projection a “UTC instant” would suggest a commitment to a competing point-based
 179 ontology. The projection is purely a computational device — a real number used to order
 180 endpoints in a single linear frame. The result's calendar and zone are taken from the first
 181 operand, preserving the caller's intent for downstream display. This first-operand-wins rule
 182 is itself a design choice that deserves community scrutiny — alternatives include rejecting
 183 heterogeneous operands, or carrying the heterogeneity through into a sum-typed result —
 184 and is one of the points the talk would invite feedback on.

185 Daylight-saving transitions.

186 The UTC projection interacts with daylight-saving transitions in two distinct ways, both han-
 187 dled at parse time so set operations on the projected values need no further disambiguation. A
 188 wall time that falls in a DST *gap* (the non-existent hour at spring-forward) is rejected at con-
 189 struction with a `ZoneGapError` — a stronger guarantee than addressing the bug, because the
 190 malformed value cannot be constructed. A wall time that falls in a DST *fold* (the doubled hour
 191 at fall-back) is disambiguated by an explicit offset in the IXDTF suffix, per RFC 9557 §4.5:
 192 the value `2024-11-03T01:30:00-04:00[America/New_York]` unambiguously picks the pre-
 193 fall-back (EDT) period and `2024-11-03T01:30:00-05:00[America/New_York]` picks the
 194 post-fall-back (EST) period. The disambiguator is part of the string representation and round-
 195 trips through serialisation — a property that distinguishes this design from meta-attribute
 196 schemes (such as Python's `fold` parameter) in which the choice is carried out-of-band and
 197 lost on serialisation.

198 Beyond the primitives.

199 Once `union`, `intersection`, and `difference` are closed, the natural extensions follow:
 200 symmetric difference, containment queries, the inverse of an interval set against a bounding
 201 context (“free time” given a calendar of meetings), and reductions across an `IntervalSet`
 202 (total duration, longest gap, densest hour).

203 5 Discussion and open questions

204 Tempo is a working library; this submission's contribution is a *design* contribution rather
 205 than a *result* contribution, and the talk would aim to make the design choices legible to
 206 an audience whose research is the theory the library implements. Three questions seem
 207 particularly worth the TIME audience's attention:

- 208 1. **Granularity and resolution.** Tempo’s next-higher-undefined-resolution rule for implicit
 209 spans is operationally simple but does not reference the more general treatment of temporal
 210 granularity in the database literature [6] nor the calendric type constraints in CaTTS [9].
 211 Is the simple rule a sufficient model in practice, or does it accumulate edge cases as the
 212 calendar set grows (lunar, sidereal, fiscal)?
- 213 2. **Allen’s algebra over interval sets.** The thirteen relations, and the $T_{\text{bounded_meeting}}$
 214 ontology that underlies them, are defined on convex intervals only [10]. Their lifting to
 215 interval sets admits several reasonable definitions (existential, universal, set-of-relations);
 216 the library currently offers the existential lifting, but the right default is not obvious,
 217 and a principled extension of the ontology to non-convex interval sets would settle the
 218 question.
- 219 3. **Cross-calendar comparison.** Comparing a Hebrew date with a Gregorian one resolves
 220 cleanly when both project to the same proleptic UTC timeline, but the timeline is
 221 itself a modelling choice (TAI? UT1? a calendar-specific civil time?). Tempo’s current
 222 choice is pragmatic; a principled framework would be a worthwhile contribution from the
 223 symposium back to the library.

224 The wider goal is to demonstrate, by existence proof, that the theoretical structure of
 225 temporal reasoning translates into an ordinary application library — one that an engineer
 226 writes `Tempo.overlaps?(a, b)` into without thinking of it as a research artefact. The talk
 227 would walk through the design, the choices made, the choices still open, and invite the
 228 symposium to sharpen them.

229 — References —

- 230 1 James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the*
 231 *ACM*, 26(11):832–843, 1983. doi:10.1145/182.358434
- 232 2 International Organization for Standardization. *ISO 8601-1:2019 — Date and time — Repre-*
 233 *sentations for information interchange — Part 1: Basic rules*, 2019.
- 234 3 International Organization for Standardization. *ISO 8601-2:2019 — Date and time — Repre-*
 235 *sentations for information interchange — Part 2: Extensions*, 2019.
- 236 4 U. Sharma and C. Bormann, editors. *Date and Time on the Internet: IXDTF Extensions*.
 237 Internet-Draft draft-ietf-sedate-datetime-extended-09, IETF, 2024. [https://www.ietf.org/](https://www.ietf.org/archive/id/draft-ietf-sedate-datetime-extended-09.html)
 238 [archive/id/draft-ietf-sedate-datetime-extended-09.html](https://www.ietf.org/archive/id/draft-ietf-sedate-datetime-extended-09.html)
- 239 5 Library of Congress. *Extended Date/Time Format (EDTF) Specification*, 2019. <https://www.loc.gov/standards/datetime/>
- 240 6 Claudio Bettini, Sushil Jajodia, and Sean X. Wang. *Time Granularities in Databases, Data*
 241 *Mining, and Temporal Reasoning*. Springer, 2000.
- 242 7 Shrey Tiwari, Serena Chen, Alexander Joukov, Peter Vandervelde, Ao Li, and Rohan Padhye.
 243 It’s about time: An empirical study of date and time bugs in open-source Python software.
 244 In *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR*
 245 *’25)*. IEEE/ACM, 2025. Distinguished Paper Award. [https://rohan.padhye.org/files/](https://rohan.padhye.org/files/datetimebugs-msr25.pdf)
 246 [datetimebugs-msr25.pdf](https://rohan.padhye.org/files/datetimebugs-msr25.pdf)
- 247 8 Giovanni Liva. *Modeling Time in Java Programs for Automatic Error Detection*. PhD thesis,
 248 Alpen-Adria-Universität Klagenfurt, 2018.
- 249 9 François Bry and Stephanie Spranger. Towards a multi-calendar temporal type system for
 250 (Semantic) Web query languages. In *Principles and Practice of Semantic Web Reasoning*
 251 *(PPSWR 2004)*, volume 3208 of *Lecture Notes in Computer Science*, pages 90–104. Springer,
 252 2004. doi:10.1007/978-3-540-30122-6_8
- 253 10 Michael Grüninger and Zhuojun Li. The time ontology of Allen’s interval algebra. In *24th*
 254 *International Symposium on Temporal Representation and Reasoning (TIME 2017)*, volume
 255

- 256 90 of *LIPIcs*, pages 16:1–16:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2017.
257 doi:10.4230/LIPIcs.TIME.2017.16
- 258 11 Curtis E. Dyreson and M. A. Manazir Ahsan. Achieving a sequenced, relational query
259 language with log-segmented timestamps. In *28th International Symposium on Temporal
260 Representation and Reasoning (TIME 2021)*, volume 206 of *LIPIcs*, pages 14:1–14:13. Schloss
261 Dagstuhl – Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPIcs.TIME.2021.14
- 262 12 Curtis E. Dyreson. Optimization of nonsequenced queries using log-segmented timestamps.
263 In *30th International Symposium on Temporal Representation and Reasoning (TIME 2023)*,
264 volume 278 of *LIPIcs*, pages 13:1–13:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik,
265 2023. doi:10.4230/LIPIcs.TIME.2023.13